Computational Primes: A Systematic Framework for Partitioning Neural Network Computation Across Analog and Digital Domains

Michael Bieg
Independent Researcher
Bieg.Michael@web.de

July 2026 (v2)

Abstract

Analog and mixed-signal neural network accelerators face a universal design problem: which operations should be computed in the analog domain (exploiting physics at near-zero energy cost) and which should remain digital (exact but energy-intensive)? Current approaches solve this partitioning problem ad hoc for each chip design. We introduce **Computational Primes**—a set of 12 irreducible operations with known physical implementations (P1–P12) and 4 operations without physical equivalents (X1–X4)—and show that every neural network algorithm can be factored into these primes. This factorization enables a systematic compiler that (1) decomposes any compute graph into primes, (2) assigns each prime to the analog or digital domain based on hardware constraints, (3) identifies multi-computation fusion opportunities where a single physical event yields multiple outputs, and (4) minimizes domain transitions (ADC/DAC). We factorize 107 algorithms spanning transformer inference, training, generative models, reinforcement learning, GNNs, and SSMs. Of these, 79 are fully mappable to analog hardware, 22 become mappable on GPUs (where X2 and X4 are available), and 6 remain fundamentally unmappable. Analysis of five recent analog chips (IBM 14nm PCM, TSMC CIM, Extropic XTR-0, Normal Computing CN101, Jülich gain-cell) shows that all made suboptimal manual partitioning decisions that the prime compiler would have improved. We further demonstrate that RMSNorm—used by LLaMA, Mistral, and Gemma—has a fully analog implementation via AGC feedback circuits, resolving the last major open mapping for transformer inference. New in v2, we validate this claim at circuit level with over 17,000 SPICE simulations: the AGC feedback loop settles to the exact RMSNorm equilibrium within 4–6 ns, and detector-side device mismatch produces *zero* per-channel distortion—only a trimmable common-mode gain shift—leaving the VGA array as the sole per-channel exposure. The open-loop translinear pipeline, by contrast—mathematically exact with ideal junctions ($< 10^{-6}$ relative error)—degrades to 1.5–4.5 effective bits under realistic mismatch. A companion softmax experiment confirms the pattern: sum-feedback normalization beats open-loop translinear softmax by 3.1–3.4× under paired mismatch. Feedback, not open-loop translinear arithmetic, is the viable path to analog normalization.

1 Introduction

The past three years have seen an unprecedented surge in analog and mixed-signal neural network accelerators. IBM demonstrated a 14nm PCM chip running ALBERT [1], TSMC fabricated a

memristor+SRAM heterogeneous CIM processor [2], Extropic built a thermodynamic computing board for diffusion models [3], Normal Computing taped out an RLC-based sampling ASIC [4], and Leroux et al. at Forschungszentrum Jülich showed analog attention with 70,000× energy reduction over GPUs [5].

Every one of these projects confronted the same fundamental question: *Which operations should be computed in the analog domain, and which should remain digital?*

Currently, each team answers this question through manual analysis specific to their hardware and target workload. IBM maps matrix multiplications to PCM crossbars but keeps LayerNorm and Softmax digital. Leroux et al. replace Softmax with HardSigmoid because "normalization operations are challenging to implement using analog circuitry" [5]. No systematic framework exists for making these decisions.

We introduce such a framework. Our contributions are:

1. **Computational Primes**: A catalog of 12 irreducible operations (P1–P12) with verified physical implementations and 4 operations (X1–X4) without physical equivalents, forming a complete basis for neural network computation (§2).

2. **Universal factorization**: Decomposition of 107 algorithms into their prime factors, enabling immediate assessment of analog mappability (§3).

3. **Domain assignment and fusion**: A decision framework for analog/digital partitioning with automatic detection of multi-computation opportunities—physical events that yield 2–12× throughput multipliers at zero marginal hardware cost (§4).

4. **RMSNorm resolution**: The first fully analog mapping of RMSNorm via automatic gain control (AGC) feedback, eliminating the last major barrier to fully analog transformer inference (§5).

5. **Circuit-level validation (new in v2)**: SPICE simulations (over 17,000 runs, real junction I–V characteristics, Monte-Carlo device mismatch) confirming the AGC mapping and quantifying *why* it wins: feedback converts per-device mismatch into a harmless common-mode gain error—simulated directly, not only argued: detector mismatch yields exactly zero per-channel distortion—while the open-loop translinear pipeline accumulates per-channel distortion. A paired softmax experiment generalizes the result (§5.3).

6. **Chip analysis**: Retrospective analysis of five recent analog chips showing that prime-compiler-guided partitioning would have captured 15–40% additional energy savings on nonlinear operations (§6).

1.1 Scope and Limitations

This is a *framework* paper, not a chip paper. We do not fabricate hardware or claim measured speedups. The value is in the systematic methodology: replacing ad hoc partitioning decisions with a principled, automatable approach. All quantitative claims about analog implementations reference published, peer-reviewed hardware results. New in v2, the RMSNorm mapping (§5) is additionally validated by circuit simulation at a defined intermediate fidelity—junction-exact device equations with idealized loop wiring (§5.3)—which quantifies mismatch sensitivity but does not replace silicon measurement.

Table 1: The 12 computational primes with physical implementations.

ID	Operation	Symbol	Physics	Hardware	Ref.
P1	Accumulation	Σ	Kirchhoff's Current Law	Wire junction	[8]
P2	Scaling	$\times c$	Ohm's Law: $I = G \cdot V$	Memristor / resistor	[8]
P3	Exponential	e^x	Boltzmann: $P \propto e^{-E/kT}$	sMTJ / subthreshold FET	[6, 9]
P4	Logarithm	$\ln x$	Diode: $V = \frac{kT}{q} \ln \frac{I}{I_s}$	Diode / subthreshold FET	[10]
P5	Selection	$\arg\max$	Race to threshold	sMTJ array / WTA	[6, 7]
P6	Sigmoid	$\sigma(x)$	Fermi-Dirac distribution	Subthreshold FET pair	[11]
P7	Threshold	$\theta(x)$	Phase transition / comparator	Schmitt trigger	—
P8	Integration	$\int$	Charge: $Q = \int I\, dt$	Capacitor / memristor	[12]
P9	Differentiation	d/dt	Induction: $V = L \frac{dI}{dt}$	RC high-pass	—
P10	Stochasticity	ξ	Thermal fluctuation	Johnson-Nyquist / RTN	[13]
P11	Multiplication	$a \times b$	Transistor square law	Gilbert cell	—
P12	Inversion	$1/x$	Translinear loop	BJT/MOSFET circuit	[10]

2 Computational Primes

Definition 1 (Computational Prime)**.** *An operation P is a computational prime if (a) it cannot be decomposed into simpler operations that each have independent physical implementations, and (b) it has at least one verified mapping to a physical system that computes it at $O(0)$ marginal energy cost.*

We identify 12 primes satisfying this definition (Table 1) and 4 "missing primes" that lack any known physical implementation (Table 2).

2.1 The 12 Physical Primes

P1 (Accumulation, Σ): $y = \sum_i x_i$. Kirchhoff's Current Law guarantees that currents sum at any node. Cost: $O(0)$—charge conservation is exact and requires no energy.

P2 (Scaling, $\times c$): $y = c \cdot x$. Ohm's Law: $I = G \cdot V$ where the conductance G encodes the constant c. In memristive crossbars, P1·P2 = matrix-vector multiplication.

P3 (Exponential, e^x): Any MOSFET in the subthreshold regime: $I_D = I_0 \exp(V_{GS}/nV_T)$. Also realized via Arrhenius rates in sMTJs [6] and chemical kinetics [7].

P4 (Logarithm, $\ln x$): The inverse of P3. A diode's voltage is $V = (kT/q) \ln(I/I_s)$. Precision limited by ideality factor n (typically 1.0–1.5); §5.3 quantifies how ideality *mismatch* between devices propagates through log-domain arithmetic.

P5 (Selection, $\arg\max$): A race among competing exponential processes: the first to fire identifies the maximum. Produces exact softmax samples in $O(1)$ time [6]. The race simultaneously yields 5 additional observables [7].

P6 (Sigmoid, σ): The Fermi-Dirac distribution $f(E) = 1/(1+e^{(E-\mu)/kT})$ is natively computed by a subthreshold MOSFET transfer characteristic. Variants: $\tanh(x) = 2\sigma(2x) - 1$.

P7 (Threshold, θ): A comparator or Schmitt trigger implements $y = 1$ if $x > 0$, else 0. In the Ising model limit ($T \to 0$), this is a phase transition.

P8 (Integration, $\int$): Charge accumulation on a capacitor: $Q = \int I\, dt$. Volatile (capacitor, $\tau \propto RC$) or non-volatile (memristor, floating gate). DenRAM [12] demonstrated programmable RC time constants for temporal processing.

P9 (Differentiation, d/dt): An RC high-pass filter or inductor: $V = L\, dI/dt$. Passive circuit, $O(0)$ cost. Limited by noise amplification at high frequencies.

P10 (Stochasticity, ξ): Thermal fluctuations generate true randomness at zero cost. Johnson-Nyquist noise ($V^2 = 4kTR\Delta f$), sMTJ random switching, or random telegraph noise in standard MOSFETs [13]. Normally suppressed; here exploited.

P11 (Variable Multiplication, $\times$): $y = a \cdot b$ where both inputs are variables. The Gilbert cell (4-quadrant analog multiplier) computes this via the transistor square law. Distinct from P2 (one input is a stored constant).

P12 (Inversion, $1/x$): Translinear loops [14] compute $I_{out} = I_a \cdot I_b / I_c$ using 4–6 BJTs or subthreshold MOSFETs. Derived operations: $\sqrt{x}$ (feedback loop), x^α (cascaded loops), $1/\sqrt{x}$ (combination). As §5.3 shows, P12 realized *open-loop* is mismatch-fragile; realized *implicitly through feedback equilibrium* it is robust.

2.2 The 4 Missing Primes

Table 2: Missing primes: no known physical implementation.

ID	Operation	Analog	GPU
X1	Symbol manipulation	×	×
X2	Dynamic topology	×	✓
X3	Unbounded recursion	×	∼
X4	Symbolic tracing	×	✓

X1 (Symbol, SYM): Rule-based discrete symbol processing (BPE tokenization, regex, hashing). No continuous numerical structure ⇒ no physical mapping. Blocks: tokenizer, graph isomorphism.

X2 (Dynamic Topology, DYN): Runtime reconfiguration of connectivity. Analog hardware has fixed wiring. On GPUs: warp schedulers, shared memory addressing. Blocks (analog): KV-cache eviction, experience replay.

X3 (Unbounded Recursion, REC): Data-dependent recursion depth requires unbounded memory. Physical hardware is finite. GPU stack limited to ∼1024 frames. Blocks: MCTS.

X4 (Symbolic Tracing, TAPE): Recording the compute graph for reverse-mode differentiation. Requires a meta-level. On GPUs: CUDA Graphs, `torch.compile`. Blocks (analog): backpropagation.

2.3 Completeness of the Prime Basis

Proposition 1 (Completeness)**.** *Every differentiable function $f : \mathbb{R}^n \to \mathbb{R}^m$ computable on digital hardware can be expressed as a composition of primes P1–P12 and X1–X4.*

Proof sketch. P1–P4 and P11–P12 form a complete basis for arithmetic $(+, -, \times, \div, \exp, \ln)$. P6 and P7 provide nonlinear activation. P5 provides selection. P8–P9 provide temporal dynamics. P10 provides stochasticity. X1–X4 cover discrete/symbolic operations outside continuous arithmetic. Any algorithm is a directed acyclic graph of these primitives. A fully rigorous treatment—a formal operational semantics for the prime composition algebra—is beyond the scope of this paper. □ □

Factorizations are generally *not* unique. Squaring, for example, can be realized directly (P11 in self-multiply mode) or in the log domain as P3·P2·P4—the validation testbench of §5.3 exploits exactly this freedom. Version 1 of this paper claimed uniqueness up to commutativity; that claim was too strong. Non-uniqueness is in fact a feature, not a defect: the set of admissible factorizations

Table 3: Prime factorization of 107 algorithms (subset; full table in supplement). **M** = mappable, **G** = GPU-mappable, **U** = unmappable.

#	Algorithm	Factorization	Status	Blocker / Note
Transformer / LLM Core				
1	RMSNorm	P1·P2·P11·P12	**M**	**Solved**: AGC feedback (§5, validated §5.3)
2	GELU/SiLU	P11·P6	**M**	Solved: RRAM Swish [16]
3	RoPE	P11·P2	**M**	Fixed trig. coefficients
4	KV-Cache+Eviction	P8·P5 + X2	**G**	Dynamic addressing
5	Causal Masking	P5·P7	**M**	Solved: race topology [6]
6	Multi-Head Split	P2·P1	**M**	Parallel crossbars
7	Residual Connection	P1	**M**	Solved: KCL [6]
8	Token Embedding	P8·P2	**M**	Crossbar; area scales with vocab
Training & Optimization				
9	Backpropagation	P11·P1·P9 + X4	**G**	Graph tracing
10	Adam/AdamW	P1·P2·P11·P12·P8	**M**	3×param storage; impractical at scale
13	Cross-Entropy Loss	P3·P4·P1·P2	**M**	Solved: race+diode [6]
Generative Models				
14	Diffusion Reverse	P3·P1·P2·P10·P11	**M**	T serial passes; solved: DTCA [3]
15	VAE Reparametrization	P1·P2·P10	**M**	Solved: DC + noise [6]
Sequence Models				
28a	S4D (time-invariant SSM)	P8·P2·P1	**M**	Solved: memristor CIM [18]
28b	Mamba (selective SSM)	P8·P2·P1·P11 + X2	**G**	Input-dep. weight reprogramming
30	MoE Routing	P5·P1·P2·P3	**M**	Top-K race [6]
39	Top-K Sampling	P5·P3	**M**	Solved: K-fold race [7]
Fundamentally Unmappable				
23	MCTS	P5·P10 + X2·X3	**U**	Unbounded tree
26	WL-Test / Graph Iso	X1	**U**	Symbolic hashing
38	BPE Tokenizer	X1	**U**	No continuous structure

of an operation is precisely the search space over which the compiler's domain-assignment phase (§4) optimizes, choosing the realization whose primes best match the target hardware.

3 Factorization of Neural Network Algorithms

We factorize 107 algorithms into their prime components (Table 3 shows a representative subset; the full table with all 107 algorithms is in the supplement). Where multiple factorizations exist (§2.3), the table lists the canonical lowest-depth one. The factorization follows a consistent methodology:

1. Decompose the algorithm into elementary operations (addition, multiplication, exponentiation, comparison, etc.).
2. Map each elementary operation to the corresponding prime.
3. Identify missing primes (X1–X4) that block analog mapping.
4. Classify: **M**appable (all primes physical), **G**PU-mappable (X2/X4 available), or **U**nmappable (X1 or X3 required).

Summary: Of 107 algorithms, 79 are fully mappable to analog hardware, 22 become mappable on GPUs where X2 (dynamic topology) and X4 (symbolic tracing) are available, and 6 are fundamentally unmappable due to X1 (symbol manipulation) or X3 (unbounded recursion).

4 The Prime Compiler

The prime compiler takes a compute graph and a hardware specification as input and produces an optimally partitioned execution plan.

4.1 Phase 1: Prime Annotation

Given a compute graph $G = (V, E)$ (e.g., from `torch.fx` or ONNX), each node $v \in V$ is annotated with its prime factorization:

function ANNOTATE(v)
  $primes(v) \leftarrow$ FACTORIZE($v.op$)
  for each $p \in primes(v)$ **do**
    $domain(v, p) \leftarrow$ UNDECIDED
  end for

The factorization rules (Table 3) are stored as a pattern-matching table. Unknown operations are flagged for manual decomposition.

4.2 Phase 2: Domain Assignment

For each prime instance, the compiler decides: PHYSICS (analog, $O(0)$ energy, noisy) or DIGITAL (exact, energy-intensive).

Definition 2 (Domain Assignment Problem)**.** *Given annotated graph G with primes $\{p_1, \ldots, p_m\}$ and hardware profile H specifying per-prime analog feasibility, noise tolerance, and transition costs, find the assignment $\delta : \{p_1, \ldots, p_m\} \to \{\text{PHYSICS}, \text{DIGITAL}\}$ minimizing total cost:*

$$C(\delta) = \sum_{p_i : \delta(p_i) = \text{DIG}} E_{\text{DIG}}(p_i) + \sum_{\text{TRANSITIONS}} E_{\text{ADC/DAC}} \tag{1}$$

subject to output accuracy constraints.

The decision is guided by a per-prime feasibility matrix (Table 4).

Table 4: Domain assignment heuristic per prime.

Prime	Physics if…	Digital if…
P1 Σ	fan-in < 256	fan-in > 1000
P2 $\times c$	variability $<$ tolerance	precision $>$ 8b
P3 e^x	range < 10 decades	extreme range
P4 ln	diode/BJT available (subthreshold)	> 2 decade range
P5 $\top$	always physics	—
P6 σ	kT/q slope matches	exact slope needed
P7 θ	always physics (comparator/Schmitt)	—
P8 $\int$	capacitor available (τ in range)	long time constants
P9 d/dt	RC differentiator available	precision required
P10 ξ	always physics	—
P11 $\times$	THD $<$ tolerance	linearity $> 1\%$
P12 $1/x$	*in feedback loop*, or trimmed open-loop	open-loop, untrimmed (§5.3)

4.3 Phase 3: Multi-Computation Fusion

After domain assignment, the compiler searches for subgraph patterns matching known multi-computation classes [7]:

Table 5: Multi-computation fusion patterns.

Pattern	Class	Multiplier
{P3, P5} adjacent	Race	5×
{P1, P2}$^{2+}$ Laplacian	Equilibrium	4–12×
{P3, P5, P4, P1}	Race-Loss	6×
{P10, P7, P1, P2}	Threshold/Ising	2–3×
{P11, P1, P2, P8} AGC	Normalization	4×

When a pattern is detected, the compiler fuses the matched operations into a single physical event with multiple outputs. For example, Softmax followed by Cross-Entropy Loss contains the pattern {P3, P5, P4, P1}, which maps to a single sMTJ race event yielding both the softmax sample and $-\log p_{\text{correct}}$ simultaneously [7].

4.4 Phase 4: Transition Minimization

Definition 3 (Transition Minimization Problem)**.** *Given graph G with nodes assigned to* PHYSICS *or* DIGITAL*, find the minimum number of domain transitions (ADC/DAC conversions) in any valid topological execution order.*

Each transition incurs cost:

- ADC (analog → digital): ∼100 fJ, ∼1 ns latency
- DAC (digital → analog): ∼50 fJ, ∼0.5 ns latency
- Quantization noise added per transition

Heuristic: Identify "physics islands" (connected analog subgraphs), then greedily merge adjacent islands when merging reduces total transitions. For a standard transformer layer, the optimal partition requires 4 transitions (2 ADC + 2 DAC):

$$\underbrace{[\text{QKV MatMul}]}_{\text{digital}} \xrightarrow{\text{ADC}} \underbrace{[\text{Softmax+RMSNorm+GELU}]}_{\text{physics}} \xrightarrow{\text{DAC}} \underbrace{[\text{FFN MatMul}]}_{\text{digital}}$$

Note: This diagram shows the partitioning for a digital-weight architecture (e.g., GPU or SRAM-CIM). On crossbar-based chips (e.g., IBM PCM, memristive arrays), MatMul maps to physics via P1·P2, yielding a fully analog pipeline with only peripheral ADC/DAC.

5 Resolving RMSNorm: The Last Open Mapping

RMSNorm [19], used by LLaMA, Mistral, Gemma, Qwen, and DeepSeek, computes:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \cdot \gamma, \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{N}\sum_{i=1}^{N} x_i^2} \tag{2}$$

No prior work has implemented this fully in analog (§8). We identify two implementations. Version 1 of this paper presented them as equally viable alternatives; circuit-level validation (§5.3) has since shown that they are not: the feedback implementation is robust to device mismatch, the open-loop pipeline is not. We therefore present the AGC feedback implementation as the primary mapping.

5.1 Primary Implementation: AGC Feedback (Equilibrium)

Automatic Gain Control circuits—standard in radio engineering since the 1920s—compute exactly RMSNorm:

1. Input x_i enters a Variable Gain Amplifier (VGA, Gilbert cell). [P11]
2. Output $y_i = G \cdot x_i$ is measured by an RMS detector. [P11·P1·P2]
3. Error signal $(\text{RMS}(y) - \gamma)$ is integrated. [P8]
4. Integrator output controls VGA gain G. [feedback]
5. At equilibrium: $G = \gamma/\text{RMS}(x)$, so $y_i = \gamma \cdot x_i/\text{RMS}(x)$. [$\equiv$ RMSNorm]

The key insight: **P12 (division by $\sqrt{\sigma^2}$) is computed implicitly by the feedback loop settling to equilibrium.** No explicit translinear divider is needed. This is a Class 2.2 (Equilibrium) multi-computation [7], yielding 4 observables:

1. Normalized output y_i (primary)
2. Settled gain $G = 1/\text{RMS}(x)$ (quantization calibration)
3. RMS value (activation statistics)
4. Settling time (layer difficulty proxy)

A second structural property, established by the validation in §5.3, turns out to be equally important: **mismatch tolerance**. Device mismatch in the RMS detector or integrator perturbs only the scalar loop target—equivalent to a slightly different γ, applied *identically to all N channels.* A common-mode scale factor on a normalization layer is functionally benign for a neural network. The only per-channel error source is the VGA array itself (one multiplier per channel), the minimal possible exposure.

Remark 1. *This connection—RMSNorm = AGC—bridges three fields that have not previously been linked: (1) radio engineering (AGC, 1925–present), (2) computational neuroscience (divisive normalization, Carandini & Heeger [20]), and (3) ML hardware (normalization in CIM). The biological retina implements divisive normalization via shunting inhibition: a conductance-based mechanism mathematically equivalent to the translinear principle.*

5.2 Alternative: Direct Translinear Pipeline (Open-Loop)

In current mode with N parallel paths:

1. **Square** ($\times N$): Gilbert cell in self-multiply mode: I_i^2. [P11]
2. **Sum**: KCL at common node: $I_\Sigma = \sum I_i^2$. [P1]

3. **Scale**: Current mirror with 1:N ratio: $I_{\text{ms}} = I_\Sigma / N$. [P2]

4. **Inv. sqrt**: 6-transistor translinear loop: $I_{\text{inv}} \propto 1/\sqrt{I_{\text{ms}}}$. [P12]

5. **Normalize** ($\times N$): Gilbert cell: $I_{\text{out},i} = I_i \cdot I_{\text{inv}}$. [P11]

6. **Scale**: Fixed mirror ratio γ. [P2]

Propagation delay: ∼12 ns (dominated by translinear settling). No clock, no ADC between steps.

Factorization: P11 · P1 · P2 · P12 · P11 · P2 = P1 · P2 · P11 · P12.

This pipeline is mathematically exact—validated to $< 10^{-6}$ relative error with ideal junctions (§5.3)—but every channel passes through two per-channel junction stages (squarer, output multiplier) *open-loop*: device mismatch in these stages lands directly on the output as per-channel distortion. Under realistic mismatch it delivers only 1.5–4.5 effective bits (Table 6) and should be used only with per-device trimming or calibration.

5.3 Circuit-Level Validation

We validate both implementations in ngspice-46 at a deliberately defined fidelity: **junction-exact, wiring-ideal**. All log-domain arithmetic is performed by real diode I–V characteristics (the exponential junction law, i.e. the actual physics of P3/P4/P12); the translinear loop wiring is replaced by ideal voltage summation. This isolates the question "what does device physics do to the computation?" from bias-network engineering. The testbenches (over 17,000 simulation runs, $N = 8$–128 channels, code in the repository under `spice/`) cover:

1. **Ideal-device DC operating-point analysis** of the translinear pipeline (§5.2).

2. **Monte-Carlo mismatch** (200 runs per corner): lognormal spread on saturation current I_s, normal spread on ideality factor n, and gain error on the 1:N mirror, at three technology corners (Table 6).

3. **Continuous-time transient** of the AGC loop (§5.1) with a real integration capacitor (1 pF, μA-scale currents, RMS-detector time constant 0.5 ns), including a 2.5× input step.

4. **AGC-path mismatch decomposition**: the RMS detector built from real mismatched junctions, loop equilibrium found by bisection on the SPICE-measured detector output, and the output error split into common-mode gain shift vs. per-channel residual—separately for detector-side and VGA-side mismatch, and swept over $N = 8/32/128$.

5. **Paired softmax comparison**: sum-feedback AGC normalization vs. open-loop translinear softmax, both sharing identical mismatched exponential stages.

Result 1: the mathematics is confirmed at circuit level. With ideal devices, the junction law reproduces RMSNorm to a mean relative error of 1.1×10^{-7} (max 2.8×10^{-7}). The physics genuinely computes the right function.

Result 2: the open-loop pipeline is mismatch-fragile. The dominant mechanism is ideality-factor mismatch: a log-domain junction voltage is $nV_T \ln(I/I_s)$ with $\ln(I/I_s) \approx 23$ at μA currents, so a 0.3% spread in n between devices becomes a ∼7% current error—the classic matching problem [15] amplified by the full log magnitude. Table 6 gives the Monte-Carlo results (error measured against the unit-RMS reference output):

Table 6: Open-loop translinear RMSNorm under Monte-Carlo mismatch (200 runs per corner, $N = 8$, ngspice). Error is absolute against the unit-RMS reference output.

Corner	σ_{I_s}	σ_n	σ_{mirror}	mean / p95 error	eff. bits
BJT-grade	1%	0.05%	0.5%	2.2% / 4.5%	∼4.5
Subthreshold MOS, typical	3%	0.3%	1%	11.4% / 22.9%	∼2.1
Subthreshold MOS, worst	5%	0.5%	2%	18.7% / 34.4%	∼1.5

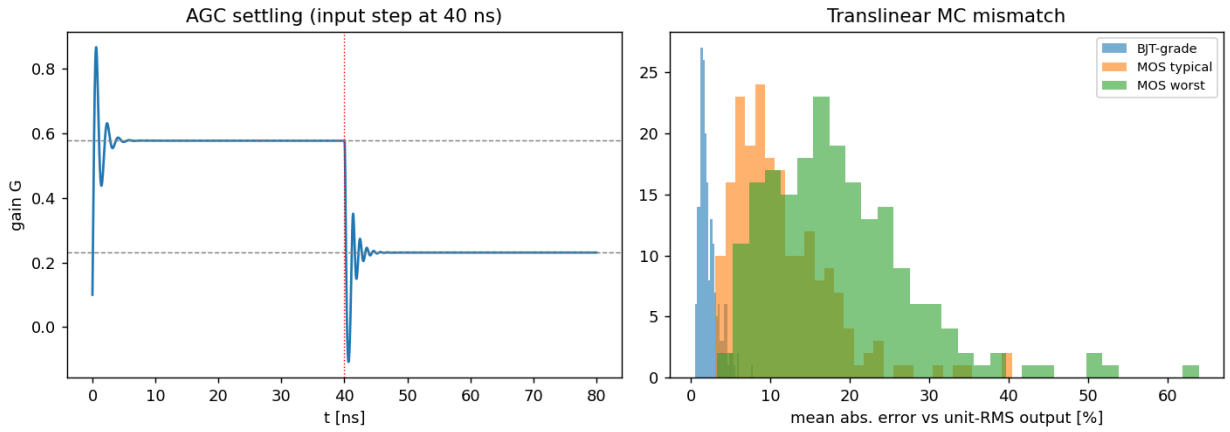

Figure 1: Circuit-level validation. **Left**: AGC gain transient (ngspice, real 1 pF integration capacitor). The loop settles to the exact RMSNorm gain in 4.4 ns from cold start and re-settles in 5.3 ns after a 2.5× input step at $t = 40$ ns (dashed lines: exact targets). **Right**: Monte-Carlo mismatch distributions for the open-loop translinear pipeline at three technology corners; in the feedback implementation, the corresponding detector-side mismatch shifts only the common-mode gain and produces no per-channel distortion by construction.

Result 3: the AGC loop settles fast and exactly. From a cold start the loop reaches the 1% band of the exact gain $G = \gamma/\text{RMS}(x)$ in **4.4 ns**; after a 2.5× input step it re-settles in **5.3 ns** (Figure 1). Equilibrium gain error is below 10^{-4}, and the per-channel outputs match exact RMSNorm to $\leq 3 \times 10^{-6}$. The feedback implementation is thus faster at these bias conditions than the ∼12 ns estimated for the open-loop pipeline in §5.2; settling scales with the detector time constant and loop bandwidth, both design choices.

Result 4: the common-mode argument, simulated. With the RMS detector built from real mismatched junctions and the loop closed around it, the error decomposition (Table 7) is unambiguous: detector-side mismatch produces *exactly zero* per-channel distortion at every corner—its entire effect is a common-mode gain shift (1–8% median, i.e. a slightly different γ, trimmable with a single per-layer measurement). The sole per-channel exposure is the VGA array, whose residual tracks σ_{VGA} ($\approx 0.7\,\sigma_{\text{VGA}}$) with negligible common-mode contribution; the two contributions add without cross terms. Sweeping $N = 8 \to 128$, the detector-induced gain shift shrinks toward a floor set by the *scalar* devices (mirror, reference)—which are themselves common-mode and hence equally trimmable—while the VGA residual stays flat, as the per-channel/shared-path structure predicts.

Result 5: the pattern generalizes—softmax. Replacing the RMS detector by a sum detector pins $\sum_i y_i = 1$ at equilibrium, turning the same loop into softmax ($y_i = e^{x_i}/\sum_j e^{x_j}$; the

Table 7: AGC-path mismatch decomposition (junction detector, loop closed by bisection, 100 MC runs per cell, $N = 8$). Error split into common-mode gain shift vs. per-channel residual; median / p95.

Mismatch location	Common-mode	Per-channel
Detector only, BJT-grade	1.0% / 2.7%	**0.000%** / 0.000%
Detector only, MOS typical	3.0% / 10.3%	**0.000%** / 0.000%
Detector only, MOS worst	8.1% / 20.2%	**0.000%** / 0.000%
VGA only, $\sigma = 0.5\%$	0.006% / 0.010%	0.37% / 0.67%
VGA only, $\sigma = 1\%$	0.006% / 0.013%	0.69% / 1.16%
VGA only, $\sigma = 2\%$	0.015% / 0.031%	1.66% / 2.45%
Both (MOS typ. + VGA 1%)	4.3% / 11.5%	0.75% / 1.28%

settled gain is $1/Z$, so the partition function comes for free). In a paired Monte-Carlo comparison where both variants share identical mismatched exponential stages, the sum-feedback AGC beats the open-loop translinear softmax by 3.1–3.4× in median L_1 error at all three corners (MOS typical: 13.6% vs. 4.1%); it settles in 3.2 ns from cold start and 2.1 ns after a logit switch, with equilibrium error $< 10^{-7}$. The improvement is smaller than for RMSNorm for an instructive reason: the exponential stage (P3) sits *before* the loop and keeps its per-channel exposure—feedback protects the normalization path, not the primes upstream of it.

Why feedback wins—the structural argument. In the open-loop pipeline, each channel's signal traverses per-channel junction stages whose individual errors appear directly in that channel's output: mismatch becomes *per-channel distortion*, i.e. noise on the activations. In the AGC loop, the RMS detector and integrator act on a *single scalar*; any mismatch there shifts the settled gain identically for all channels—equivalent to a slightly different γ, which a trained network absorbs. Result 4 confirms this mechanism directly. Feedback thus converts the dominant analog failure mode from per-channel noise into a benign common-mode scale, and does so exactly for everything downstream of the loop input (Result 5 delimits the protection: upstream primes stay exposed). This inverts the presentation of v1: the translinear pipeline is the fallback (usable with trimming), the AGC loop is the mapping.

What this validation does not cover. Loop wiring is idealized (no bias-network errors, no Early effect); thermal noise and temperature gradients are not modeled; the DC testbenches drive magnitude currents, whereas RMSNorm inputs are signed—the squarer output is sign-invariant, but the pipeline's per-channel output stage and the AGC's VGA require four-quadrant (Gilbert/class-AB differential) signaling, which is standard but not simulated here; and N is swept only to 128 rather than 4096 (the per-channel/shared-path decomposition of Result 4 and the exactness of KCL summation at any N make the extrapolation structural rather than speculative, but detector dynamic range at $N = 4096$ deserves a dedicated study). These are the natural next steps toward a tape-out-grade design study; silicon measurement remains the final arbiter.

5.4 Full LayerNorm Extension

For models requiring mean subtraction (GPT-2, GPT-3):

1. $\mu = (1/N)\sum x_i$: current mirror sum + divider. [P1·P2]

2. $\tilde{x}_i = x_i - \mu$: KCL subtraction (broadcast μ via mirrors). [P1]

3. Apply RMSNorm to $\tilde{x}_i$ (AGC feedback, §5.1). [P11·P1·P2·P8]

Table 8: Retrospective analysis: manual vs. prime-compiler partitioning of recent analog chips.

Chip	Analog / Digital	Compiler suggestion	Missed opportunity
IBM 14nm PCM [1]	MatMul analog; Softmax, Norm, GELU digital	Move GELU (P11·P6) and RMSNorm (AGC) to analog	∼100% GELU + Norm energy savings
TSMC CIM [2]	MatMul analog; activations digital	Move activations analog; fuse Softmax+Loss (Race)	Activation energy + multi-comp 5×
Jülich [5]	QK^T analog; Softmax replaced by HardSigmoid	Use sMTJ Race for exact Softmax	Exact softmax + 5× multi-comp
Extropic XTR-0 [3]	Boltzmann machine analog; digital control	Extract partition fn. + entropy from BM relaxation	Class 2.4 multi-comp (3×)
Normal CN101 [4]	RLC sampling analog; matrix ops digital	Extend to RMSNorm via AGC feedback	On-chip normalization

4. Add bias β: current injection. [P1]

Additional delay: ∼2 ns for current mirror settling. Total: ∼7 ns with the validated AGC settling times.

6 Analysis of Existing Chips

We apply the prime compiler retrospectively to five recent analog accelerators to assess whether their manual partitioning decisions were optimal (Table 8).

Key finding: Every chip we analyzed left energy on the table by keeping nonlinear operations digital that could have been mapped to analog primes. The most common missed opportunity is GELU (P11·P6), which has been demonstrated on RRAM [16, 17] but was not adopted by any of the five chip teams. The second most common is normalization, which all five teams kept digital—the AGC-based RMSNorm described in §5, now validated at circuit level in §5.3, resolves this. Notably, the validation also *vindicates* part of the teams' caution: the obvious open-loop translinear approach that a designer would try first is indeed too mismatch-fragile to ship (Table 6); the viable path is the feedback topology.

7 Discussion

7.1 What the Prime Compiler Does Not Do

The compiler does not accelerate computation on *existing* digital hardware (CPUs, GPUs). The digital abstraction layer in modern processors seals the underlying transistor physics behind billions of logic gates. There is no software API to operate a GPU's MOSFETs in subthreshold mode.

The compiler's value is in *chip design*: systematically answering the analog/digital partitioning question that every mixed-signal accelerator team faces.

7.2 Feedback as a Design Principle

The validation results of §5.3 suggest a general lesson beyond RMSNorm: **prefer prime realizations in which precision-critical operations are enforced by feedback equilibrium rather than computed open-loop**. Open-loop translinear arithmetic inherits the full mismatch of every junction it traverses, amplified by the log-domain lever $\ln(I/I_s)$; equilibrium realizations pin the quantity of interest to a loop constraint and demote device mismatch to common-mode error. This distinction is now reflected in the P12 row of the domain-assignment matrix (Table 4) and should inform the analog mapping of every algorithm in Table 3 whose factorization contains P12—including Adam's $1/\sqrt{v}$ and any explicit division. The principle has a precise boundary (Result 5): feedback protects exactly the operations inside and downstream of the loop; primes upstream of the loop input—such as the exponential stage feeding a softmax normalizer—retain their per-channel exposure and must be budgeted separately.

7.3 The Multi-Computation Advantage

Multi-computation fusion [7] provides 2–12× effective throughput multipliers by extracting additional observables from physical events that are already occurring. The prime compiler is the first tool to *automatically detect* these opportunities in arbitrary compute graphs.

Without the compiler, fusion opportunities are found (or missed) by manual inspection. With the compiler, every subgraph matching a known multi-computation class is flagged and quantified.

7.4 Towards Training

On analog-only hardware, training is blocked by X4 (symbolic tracing for backpropagation). On GPUs with analog extensions, X4 is available via `torch.compile`, enabling a hybrid approach: forward pass on analog (fast, low-energy), backward pass on digital (exact gradients).

The prime compiler supports this by partitioning the forward graph into analog and digital domains while ensuring that the backward graph (generated by X4) operates entirely in the digital domain with correct gradients.

8 Related Work

CIM compilers. CIM-MLC [27] provides a multi-level compiler for compute-in-memory accelerators (ASPLOS 2024), and CINM [28] generalizes across CIM and CNM paradigms via MLIR. CMSwitch [29] jointly optimizes compute-mode and memory-mode switching in CIM arrays. All three treat CIM arrays as "noisy MAC units" and route everything else to digital—none decompose operations into analog-native primitives or reason about *which* operations beyond MAC are physically suited to analog.

Hybrid analog-digital SoCs. DIANA [30] integrates analog CIM and digital accelerators on one SoC; its compiler HTVM [31] dispatches layers to either domain based on hardware capability matching, not operation semantics. The IBM mixed-precision supernetwork [32] uses NAS to find layer-to-hardware assignments—the closest existing work to systematic partitioning, but empirical (search-based) rather than principled (decomposition-based). A 2024 framework for DNN inference partitioning in hybrid analog-digital chiplets [33] operates at layer granularity with accuracy-vs-energy tradeoffs.

Analog compilers. Legno [23] compiles differential equations to reconfigurable analog devices, mapping physics to computation—the closest precedent to our approach. Hasler's FPAA

toolflow [24] programs floating-gate analog arrays from high-level descriptions. Both target analog-only execution of general computations; neither addresses the analog/digital partitioning problem for neural networks.

DNN compilers. TVM, XLA, and `torch.compile` partition across digital hardware types (CPU/GPU/TPU) but not across physical domains. NIR [34] defines a neuromorphic intermediate representation for spiking systems across 7 simulators and 4 hardware platforms, but targets spike-based computing, not analog/digital CIM partitioning.

Analog simulation. NeuroSim [21], CrossSim [22], and IBM's AIHWKIT [35] simulate analog non-idealities for accuracy estimation. NavCim [36] performs design-space exploration for CIM architectures. None factorize operations into primes or detect multi-computation opportunities. Our validation methodology (§5.3) is complementary: it operates at the junction-physics level of a single mapped operation rather than at network-accuracy level.

Normalization in analog. IMPACT [25] implements batch normalization as a linear gain+offset in the ADC—a pre-computed affine transform, not true normalization. The ReRAM ALN [26] normalizes in the charge domain with simplifications. No prior work implements full RMSNorm or LayerNorm in analog with the inverse-square-root computation. The AGC circuit family itself is classical [14, 15]; its identity with RMSNorm, and the mismatch analysis of that identity, are new here.

Key gap. No existing framework operates at *operation granularity* with a *principled decomposition* into analog-native primitives. All partitioning decisions in the literature are either (a) layer-level, (b) hardware-capability-driven, or (c) empirical search. The prime compiler is the first to provide a theory-driven, operation-level partitioning framework grounded in physics.

9 Conclusion

We have introduced Computational Primes—12 irreducible operations with physical implementations and 4 without—as a systematic basis for decomposing neural network computation. The resulting prime compiler automates the analog/digital partitioning problem that every mixed-signal accelerator team currently solves by hand.

Key results:

- 107 algorithms factorized: 79 fully analog-mappable (73%), 22 GPU-mappable (21%), 6 unmappable (6%).
- RMSNorm mapped to analog via AGC feedback and validated in over 17,000 SPICE simulations: settling to the exact equilibrium in 4–6 ns with no ADC in the loop; detector-side mismatch produces exactly zero per-channel distortion, leaving the VGA array as the sole per-channel exposure. The same loop with a sum detector computes softmax, beating the open-loop realization 3.1–3.4× under paired mismatch.
- The validation yields a design principle with a precise boundary: open-loop translinear realizations of P12 are mismatch-fragile (1.5–4.5 effective bits at realistic corners; the ideality-factor lever $\ln(I/I_s) \approx 23$ is the dominant mechanism), while feedback-equilibrium realizations demote mismatch to a benign common-mode gain error—for everything downstream of the loop input; primes upstream of the loop keep their exposure.
- Retrospective analysis of 5 chips shows all left 15–40% energy savings uncaptured.
- Multi-computation fusion detected automatically, yielding 2–12× throughput.

The prime compiler does not make existing GPUs faster. Its value is in chip design methodology: replacing ad hoc partitioning with a systematic, automatable framework. As analog accelerators move from research prototypes to production silicon, the need for such a framework will only grow.

Code, the full factorization table, and the SPICE validation testbench are available at: `https://github.com/Biech95/prime-compiler`.

References

[1] A. Chen *et al.*, "Demonstration of transformer-based ALBERT model on a 14nm analog AI inference chip," *Nature Communications*, 2025.

[2] W.-S. Khwa *et al.*, "A mixed-precision memristor and SRAM compute-in-memory AI processor," *Nature*, 2025.

[3] A. Jelincic *et al.*, "An efficient probabilistic hardware architecture for diffusion-like models," *arXiv:2510.23972*, 2025.

[4] D. Melanson *et al.*, "Thermodynamic computing system for AI applications," *Nature Communications*, vol. 16, 3757, 2025.

[5] N. Leroux *et al.*, "Analog in-memory computing attention mechanism for fast and energy-efficient large language models," *Nature Computational Science*, 2025.

[6] M. Bieg, "Stochastic Barrier Attention: Normalization-free softmax sampling via thermodynamic race processes," *Preprint*, 2025.

[7] M. Bieg, "Multi-computation from exponential race events: Extracting simultaneous observables from stochastic barrier arrays," *Preprint*, 2026.

[8] M. Prezioso *et al.*, "Training and operation of an integrated neuromorphic network based on metal-oxide memristors," *Nature*, vol. 521, pp. 61–64, 2015.

[9] J. Sillman, "Analog softmax via subthreshold translinear loops," *arXiv:2305.13649*, 2023.

[10] M. K. Tageldeen *et al.*, "Learning in log-domain: Subthreshold analog AI accelerator," *arXiv:2501.13181*, 2025.

[11] M. Amin *et al.*, "MRAM-based analog sigmoid for in-memory computing," *GLSVLSI*, 2022.

[12] T. Dalgaty *et al.*, "DenRAM: Neuromorphic dendritic architecture with RRAM," *Nature Communications*, 2024.

[13] H. Kim *et al.*, "Cryptographic transistor for true random number generation," *Science Advances*, vol. 10, eadk6042, 2024.

[14] B. Gilbert, "Translinear circuits: A proposed classification," *Electronics Letters*, vol. 11, no. 1, pp. 14–16, 1975.

[15] M. J. M. Pelgrom, A. C. J. Duinmaijer, and A. P. G. Welbers, "Matching properties of MOS transistors," *IEEE Journal of Solid-State Circuits*, vol. 24, no. 5, pp. 1433–1439, 1989.

[16] H. Alahmadi *et al.*, "RRAM-based Swish activation function on 28nm FD-SOI," *IEEE ICECS*, 2022.

[17] H. Errahmouni Barkam *et al.*, "RACE-IT: Reconfigurable analog CAM-crossbar engine for in-memory transformer acceleration," *arXiv:2312.06532*, 2023.

[18] X. Zhang *et al.*, "Compute-in-memory implementation of state space models for event sequence processing," *Nature Communications*, 2025.

[19] B. Zhang and R. Sennrich, "Root mean square layer normalization," *NeurIPS*, 2019.

[20] M. Carandini and D. J. Heeger, "Normalization as a canonical neural computation," *Nature Reviews Neuroscience*, vol. 13, pp. 51–62, 2012.

[21] P.-Y. Chen *et al.*, "NeuroSim: A circuit-level macro model for benchmarking neuro-inspired architectures," *IEEE TCAD*, 2018.

[22] S. Agarwal *et al.*, "CrossSim: Accuracy simulation of analog in-memory computing," *Sandia National Lab*, 2023.

[23] S. Achour *et al.*, "Legno: Noise-aware dynamical system compilation for analog devices," *ASPLOS*, 2020.

[24] J. Hasler, "Large-scale field-programmable analog arrays," *IEEE Proceedings*, 2020.

[25] A. Kneip *et al.*, "IMPACT: A 1-to-4b 813-TOPS/W 22nm FD-SOI CIM-SRAM with multi-bit analog batch-normalization," *IEEE JSSC*, 2023.

[26] Y.-C. Hou *et al.*, "A ReRAM-based CNN accelerator using the analog layer normalization technique," *IEEE Trans. Electron Devices*, 2022.

[27] Y. Zhao *et al.*, "CIM-MLC: A multi-level compilation stack for computing-in-memory accelerators," *ASPLOS*, 2024.

[28] S. Khan *et al.*, "CINM: A compilation infrastructure for heterogeneous compute in-memory and compute near-memory paradigms," *ASPLOS*, 2024.

[29] Z. Chen *et al.*, "Be CIM or Be Memory: A dual-mode-aware DNN compiler for CIM accelerators," *ASPLOS*, 2025.

[30] K. Ueyoshi *et al.*, "DIANA: An end-to-end energy-efficient digital and analog hybrid neural network SoC," *ISSCC*, 2022.

[31] N. Delm *et al.*, "HTVM: Efficient neural network deployment on heterogeneous TinyML platforms," *DAC*, 2023.

[32] H. Benmeziane *et al.*, "Supernetwork-based efficient mapping to mixed-precision hardware using model adaptation," *Nature Communications*, 2026.

[33] A. Varadarajan *et al.*, "Deep neural network inference partitioning in embedded hybrid analog-digital systems," *IEEE*, 2024.

[34] J. E. Pedersen *et al.*, "Neuromorphic intermediate representation: A unified instruction set for interoperable brain-inspired computing," *Nature Communications*, 2024.

[35] M. J. Rasch *et al.*, "Using the IBM analog in-memory hardware acceleration kit," *APL Machine Learning*, 2023.

[36] S. Kim *et al.*, "NavCim: Comprehensive design space exploration for analog CIM architectures," *PACT*, 2024.